1. Corpus

This system is built using the **Enron Email Dataset** sourced from the public Kaggle release. The dataset contains real-world internal corporate communications exchanged between Enron employees across departments and roles. It provides a realistic setting for modeling organizational memory, including operational coordination, requests, planning discussions, and decision-making workflows.

The dataset was selected for three main reasons. First, it contains naturally occurring business communication aligned with the objective of extracting structured organizational memory. Second, the conversational and threaded nature of the emails supports temporal and relational reasoning across interactions. Third, its public availability ensures reproducibility and transparency.

For this project, we used the Kaggle CSV release containing raw RFC822-formatted emails (data/emails.csv). A custom preprocessing pipeline was implemented to parse and normalize messages into structured JSON format. This included:

- parsing sender and recipient fields using email header utilities
- extracting clean plaintext bodies from multipart messages
- normalizing timestamps into structured datetime format
- generating thread identifiers via normalized subject hashing
- removing duplicate emails using both message ID matching and content fingerprinting

After preprocessing and deduplication, we used a cleaned subset of **656 emails** for downstream extraction and graph construction.

Reproducibility

To reproduce the pipeline:

1. Download the Enron Email Dataset from Kaggle (https://www.kaggle.com/datasets/wcukierski/enron-email-dataset/data)
2. Parse and clean emails into structured JSON(top 1000 rows):
   python src/inspect_data.py
3. Build the memory graph:
   python src/build_graph.py
4. Query the system:
   python src/query_memory.py

5. Launch visualization:
   streamlit run src/visualize_memory.py

2. Structured Extraction

This section describes the system used to extract structured, typed, and grounded memory objects from unstructured email text. The focus is not only on model usage, but on the full extraction system, including schema design, grounding requirements, validation mechanisms, and safeguards that ensure extraction outputs remain reliable as the corpus evolves.

Ontology / Schema Design

We designed a lightweight but extensible schema tailored to organizational email workflows. The system models two primary object types:

Entities

The only entity type explicitly modeled is:

- **Person** — derived directly from email sender and recipient fields.

This design keeps the ontology simple while supporting actor-centric reasoning across communications.

Claim Types

From each email, the system extracts at most one structured claim representing the most important business-relevant action or decision statement.

We define five claim types:

- **RequestClaim** — the sender asks someone to perform a task
- **CommitmentClaim** — the sender commits to an action
- **ProposalClaim** — the sender proposes a plan or strategy
- **MeetingAnnouncementClaim** — the sender schedules or announces a meeting
- **InformationShareClaim** — the sender shares important operational information

This schema is implemented in:
src/claim_extractor.py → build_prompt()

where the ontology is enforced through constrained prompting and structured output requirements.

Grounding

Every extracted claim is explicitly grounded in source evidence to ensure traceability and auditability.

Grounding is implemented in:
src/claim_extractor.py → extract_claims()

Each claim stores:

- **source id** — email message_id
- **excerpt** — exact extracted text span
- **timestamp** — email timestamp
- **thread id** — normalized conversation grouping

This guarantees that every structured memory unit remains verifiable against its original source.

Validation & Repair

Because LLM outputs are inherently probabilistic, the system implements deterministic safeguards to maintain correctness.

Validation is implemented in:
src/claim_extractor.py → extract_claims()

The pipeline performs:

- strict JSON parsing
- fallback regex recovery if parsing fails
- schema enforcement (required fields present)
- minimum-length filtering for low-information outputs

Invalid or partial outputs are discarded rather than forced into memory.
This ensures robustness against malformed or inconsistent model responses.

Versioning

Although lightweight for this assignment, the extraction pipeline is designed to support version tracking across:

- prompt design
- model selection
- schema evolution

Version control is managed implicitly through:

- repository version history
- prompt definitions in claim_extractor.py
- reproducible pipeline scripts

If the ontology evolves, the system supports backfilling by re-running extraction over stored emails using updated schema definitions.

Quality Gates

To prevent noisy extractions from becoming durable memory, multiple quality gates are enforced:

1. **Heuristic filtering (pre-LLM)** — implemented in: src/build_graph.py

   Only emails containing action-oriented signal terms are processed.

2. **Confidence scoring (post-extraction)** — implemented in: src/build_graph.py

   Confidence considers linguistic strength, modality, and temporal specificity.

3. **Minimum-information constraints**

   Short or conversational outputs are discarded during validation.

Together, these safeguards ensure only high-quality, grounded claims become durable memory.

3. Deduplication & Canonicalization

A memory system must prevent redundant storage while preserving traceability and correctness. This system implements deduplication across multiple levels — messages, entities, and claims — while ensuring merges remain explainable and reversible.

Artifact Deduplication

Artifact-level deduplication prevents repeated ingestion of identical or near-identical emails caused by forwarding, quoting, or dataset redundancy.

This is implemented during preprocessing in:
src/inspect_data.py → compute_fingerprint()

Each parsed email is assigned a deterministic fingerprint generated from:

- sender
- recipients
- subject
- body content

Fingerprints are computed using:
hashlib.sha256()

Duplicate emails are detected during batch parsing in:
src/inspect_data.py → parse_batch()

and removed before downstream processing. This ensures only unique messages enter the memory pipeline.

Entity Canonicalization

Entities in this system are represented primarily as email addresses extracted during message ingestion in:
src/build_graph.py → MemoryGraph.add_message()

Using canonical email addresses avoids most ambiguity introduced by name variation.|
For broader production systems, canonicalization would additionally include:

- alias resolution (e.g., display names vs addresses)
- organizational directory alignment
- embedding-based clustering for unresolved collisions

These extensions would ensure consistent representation across heterogeneous data sources.

Claim Deduplication

Claim-level deduplication merges repeated statements of the same fact while preserving supporting evidence.

This is implemented in:
src/build_graph.py → MemoryGraph.add_claim()

Claims are normalized via:

- lowercase transformation
- whitespace trimming

Identity is defined deterministically as:
actor + normalized_content and hashed

If a duplicate claim is detected:

- no new claim node is created
- evidence is appended to the existing claim

This ensures the system stores one canonical memory object per fact while maintaining multi-source grounding.

Conflicts and Revisions

The current system does not explicitly model temporal conflicts or decision reversals. Claims are stored as atomic observations grounded in their original timestamp.

However, the system design supports extension through:

- validity intervals (*valid_from / valid_to*)
- supersession relationships between claims
- confidence-weighted temporal ranking during retrieval

These mechanisms would allow the system to represent evolving truths (e.g., ownership changes, decision reversals) in future iterations.

Reversibility

Merges in the current system remain reversible due to deterministic identity rules and preserved evidence trails.

Each merged claim maintains:

- all supporting evidence
- original timestamps
- source message identifiers

Because merges are rule-based and non-destructive, reversing a merge would simply require recomputing claim identity under revised rules.

For production deployments, reversibility would additionally include:

- merge logs
- human audit trails
- manual override workflows

```
PS D:\Github\Repository\layer10-enron-memory> python -u "d:\Github\Repository\layer10-enron-memory\src\inspect_data.py"

Total processed: 1000
Unique emails: 656
Duplicates removed: 344

Saved processed emails to data/processed_emails.json
```

4. Memory Graph Design

The system represents long-term organizational memory as a structured, grounded graph where claims form the central memory unit and are linked to entities, source artifacts, and supporting evidence. The design prioritizes traceability, queryability, and maintainability over time.

Core Objects

The memory graph is implemented as an adjacency-style in-memory graph using Python objects in:
src/build_graph.py → class MemoryGraph

The core objects are:

Entities

Entities represent organizational actors (primarily individuals identified by email address). They are created during ingestion in:
MemoryGraph.add_message()

and stored in:
self.people

Edges track communication relationships:

- self.sent_edges (actor → message)
- self.received_edges (recipient → message)

Artifacts (Messages)

Emails function as source artifacts and are stored in:
self.messages

Each artifact includes:

- message_id
- sender/recipients
- timestamp
- subject/body
- thread_id

Thread grouping is computed via:
normalize_subject()

and hashed subject keys.

Claims (Primary Memory Unit)

Claims represent durable memory objects extracted from emails.
They are stored in:
self.claims

and created via:
MemoryGraph.add_claim()

Each claim includes:

- type
- actor
- targets
- normalized content
- confidence score
- supporting evidence list

Graph relationships are maintained using:

- self.claim_actor_edges
- self.claim_target_edges

Evidence Pointers

Every claim contains grounding metadata including:

- message_id
- timestamp
- excerpt
- thread_id

These are stored inside:
claim["evidence"]

ensuring traceability back to the original artifact.

Optional Summaries

The system does not currently store persistent summaries.
However, contextual summaries are dynamically constructed during retrieval in:
MemoryGraph.build_context_pack()

This avoids summary drift while preserving interpretability.

Time Representation

The system currently models:
event time (email timestamp) stored within claim evidence records.

Validity time ("when a claim stops being true") is not explicitly modeled but is supported conceptually via:

- multiple evidence timestamps
- recency-aware retrieval ranking

Current relevance is determined using:
MemoryGraph.compute_recency_score()

which biases toward newer evidence during retrieval.

Updates & Incremental Ingestion

The design supports safe incremental ingestion:

Idempotency

Claims are deterministically identified via:
actor + normalized_content

hashed using SHA256.
This prevents duplication across repeated runs.

Reprocessing

The system supports full reprocessing by rebuilding the graph from artifacts using:
src/build_graph.py (main execution flow)

Handling Edits/Deletes/Redactions (Conceptual)

While not currently implemented, the architecture supports:

- soft deletion flags
- evidence invalidation
- claim supersession links

These mechanisms would allow safe correction workflows.

Permissions (Conceptual Design)

The current prototype assumes open access to all artifacts.
However, a production deployment would enforce permissions through:

- artifact-level access control
- claim filtering based on accessible evidence
- role-based retrieval constraints

Because claims remain grounded to source artifacts, permission filtering can occur at query time.

Observability

The system tracks extraction and memory quality through:

Execution Logging

Processing transparency is provided via terminal logs in:
src/build_graph.py (main loop)

including:

- email progress
- extraction outcomes
- processing time

Graph-Level Metrics

System health can be assessed using:
MemoryGraph.summary()

which reports:

- total entities
- messages
- claims
- edges

Future Monitoring Extensions

A production system would track:

- claim acceptance rates
- confidence drift
- duplicate frequency
- retrieval satisfaction metrics

to detect degradation over time.

5. Retrieval & Grounding

The memory graph supports grounded question answering through a structured retrieval pipeline that maps natural language queries to ranked claims and returns an evidence-backed context pack. Retrieval is implemented as a lightweight script-based interface in:
src/query_memory.py

using graph functionality defined in:
src/build_graph.py → class MemoryGraph

Mapping Questions to Candidate Claims

The system uses a hybrid retrieval strategy combining:

Entity Anchoring

When an actor is specified (e.g., “Phillip”), retrieval begins with:
MemoryGraph.retrieve_context(actor)

This restricts search to claims linked to the selected entity via:
self.claim_actor_edges

This reduces noise and ensures relevance.

LLM-Assisted Relevance Filtering

Candidate claims are passed to a lightweight relevance selection step implemented in:
MemoryGraph.answer_question()

The model is prompted with:

- the question
- numbered candidate claims

and returns only the indices of relevant claims.
This allows flexible semantic matching without full embedding infrastructure.

Expansion and Ranking Strategy

To avoid uncontrolled expansion, retrieval is bounded through:

Confidence Filtering

Claims below threshold are excluded via:
min_confidence parameter (default = 0.5)

Recency-Aware Ranking

Claims are ranked using a combined score:
0.7 × confidence + 0.3 × recency

implemented in:
MemoryGraph.retrieve_context()

Recency is computed using:
MemoryGraph.compute_recency_score()

This ensures newer, higher-confidence claims dominate results.

Result Limiting

Returned context is capped via:
limit parameter (default = 5)

preventing context explosion.

Grounded Context Construction

All retrieval outputs are fully grounded through explicit evidence links.

Context packs are constructed in:
MemoryGraph.build_context_pack()

Each retrieved item includes:

- claim text
- confidence score
- actor/targets
- message_id
- timestamp
- excerpt

This ensures every returned memory is traceable to source artifacts.

Citation Format

Final answers require explicit citations and are generated using:
MemoryGraph.answer_question()
The response prompt enforces:

- grounded reasoning
- explicit Message-ID citation
- rejection of unsupported inference

This prevents hallucinated outputs.

Ambiguity and Conflicting Evidence

The system handles ambiguity conservatively.

Multiple Valid Sources

If multiple claims are relevant:
all grounded evidence is shown.

Insufficient Evidence

If context does not support the question:

system returns: Insufficient grounded information.

Future Extensions

Conflict handling could be expanded through:

- temporal validity modeling
- claim contradiction detection
- follow-up clarification prompts

```
PS D:\Github\Repository\layer10-enron-memory> python -u "d:\Github\Repository\layer10-enron-memory\src\query_memory.py"
Memory graph loaded.

Ask a question (or type 'exit'): What meetings did Phillip schedule or ask others to attend?

DEBUG RELEVANCE RESPONSE:
 3, 1, 2, 5 (these claims involve scheduling or asking others to attend meetings.)

Retrieved Evidence:

        1. Please plan to attend the Var, Reporting and Resources Meeting on Wednesday, October 11th from 2:30 - 3:30 at EB30C1
        Confidence: 0.95
        Message: <19773657.1075855687649.JavaMail.evans@thyme>
        Date: 2000-10-04 09:23:00-07:00


        2. There will be a meeting on Tuesday, Oct. 10th at 4:00pm in EB3270 regarding Storage Strategies in the West.
        Confidence: 0.95
        Message: <29177675.1075855687692.JavaMail.evans@thyme>



        3. I have scheduled and entered on each of your calendars a meeting for the Gas Physical/Financial Position. It will take place on Thursday, 9/28 from 3:00 - 4:00 in Room
 EB2537.
        Confidence: 0.95
        Message: <24741229.1075855687937.JavaMail.evans@thyme>
        Date: 2000-09-26 05:07:00-07:00


        4. There will be a Friday Meeting August 18, 2000. This meeting replaces the every Friday Meeting and will be held every other Friday. Date: Friday August 18, 2000, Time:
 2:30 pm - 4:30 pm, Location: 30C1, Topic: ENA Management Committee.
        Confidence: 0.95
        Message: <13302421.1075855689330.JavaMail.evans@thyme>
        Date: 2000-08-16 03:59:00-07:00


Answer:
 Based on the provided context, Phillip scheduled or asked others to attend the following meetings:

1. Var, Reporting and Resources Meeting on Wednesday, October 11th from 2:30 - 3:30 at EB30C1 (Message ID: <19773657.1075855687649.JavaMail.evans@thyme>)
2. Meeting on Tuesday, Oct. 10th at 4:00pm in EB3270 regarding Storage Strategies in the West (Message ID: <29177675.1075855687692.JavaMail.evans@thyme>)
3. Gas Physical/Financial Position Meeting on Thursday, 9/28 from 3:00 - 4:00 in Room EB2537 (Message ID: <24741229.1075855687937.JavaMail.evans@thyme>)
4. Friday Meeting on August 18, 2000. This meeting replaces the every Friday Meeting and will be held every other Friday. Date: Friday August 18, 2000, Time: 2:30 pm - 4:30 pm, L
ocation: 30C1, Topic: ENA Management Committee (Message ID: <13302421.1075855689330.JavaMail.evans@thyme>)

Insufficient grounded information to determine if there were any other meetings scheduled or asked to attend by Phillip.
```

6. Visualization Layer

To make extracted memory explorable, we implemented two complementary visualization approaches:

1. an interactive graph visualization for structural exploration
2. a Streamlit-based web interface for investigative workflows

Both operate over the serialized graph stored in:
memory_graph.pkl

1. Interactive Graph Visualization (PyVis)

We implemented a structural visualization of the memory graph using:
src/visualize_graph.py

This script loads the serialized graph and renders an interactive network using:
pyvis.network.Network

Graph Structure

The visualization represents:

- **actors (people)** as blue nodes
- **claims** as green nodes
- **emails (evidence artifacts)** as red nodes

Edges represent:

- actor → claim relationships
- claim → supporting evidence links

This directly mirrors the graph structure implemented in:
src/build_graph.py → class MemoryGraph

Exploration Features

The PyVis interface supports:

- zoom + pan exploration
- hover inspection for metadata
- interactive layout physics

Each node displays contextual metadata:

- claims show content + confidence
- evidence nodes show timestamp + excerpt

The graph is exported as:
memory_graph_visualization.html
for browser-based exploration.

Design Role

This visualization is best suited for:

- structural inspection
- relationship discovery
- understanding graph topology

However, large graph layouts became visually dense, which motivated the second interface.

2. Investigative Web Interface (Streamlit)

We implemented a task-focused visualization layer using:
src/visualize_memory.py

This interface enables stepwise exploration:
entity → claim → evidence
which better supports real-world investigative workflows.

Filtering & Navigation

The sidebar provides dynamic filtering by:

- actor
- claim type
- confidence threshold
- time (year)

Filtering operates directly over:
graph.claim_actor_edges
graph.claims
allowing targeted exploration without overwhelming the user.

Claim Exploration Workflow

The interface provides two levels of interaction:

Overview Table

Displays:

- claim type
- truncated content
- confidence score

for rapid scanning.

Detailed Claim View

Selecting a claim reveals:

- full claim text
- confidence bar
- structured metadata

Evidence Panel

Each claim includes a fully grounded evidence panel showing:

- message ID
- timestamp
- thread ID
- exact excerpt

This ensures every memory unit remains verifiable against source artifacts.

Duplicate & Merge Inspection

Merged claims remain explorable through:

- multi-evidence listings
- shared canonical claim identity

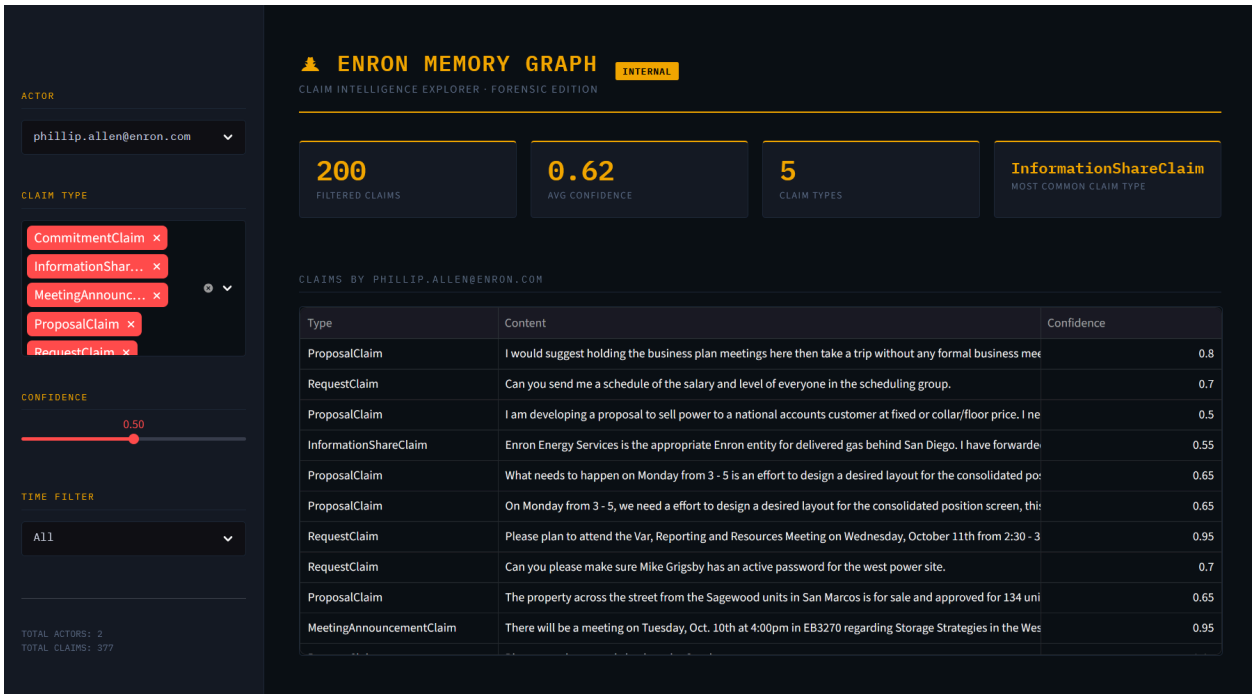
Users can inspect all supporting sources via the evidence panel.

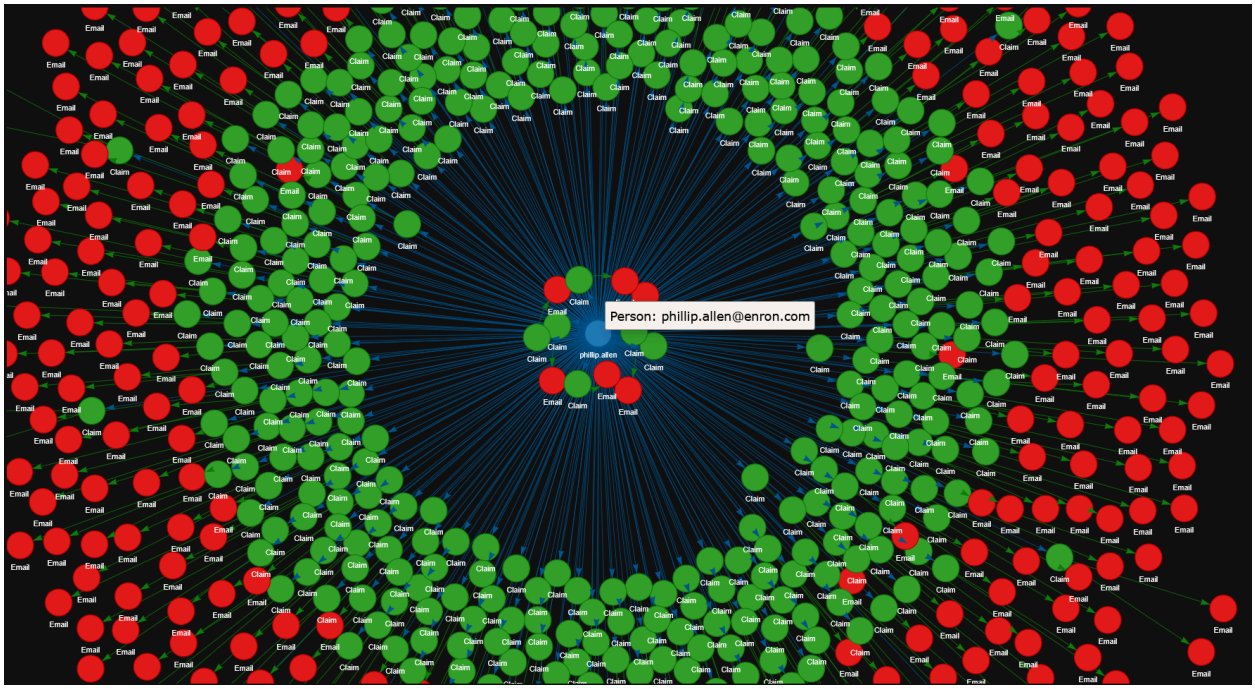
Design Rationale

The two visualization modes serve different purposes:

- **PyVis graph** → structural exploration
- **Streamlit UI** → grounded investigation

Together, they provide both macro-level understanding and micro-level explainability.





7. Layer10 Considerations

Adapting this system to Layer10's production environment would require extending the ontology, strengthening grounding guarantees, and supporting cross-system integration across email, chat platforms, documents, and structured systems like Jira or Linear. The core architecture remains applicable, but several components would evolve.

Unstructured + Structured Fusion

In Layer10's environment, memory must unify discussions across:

- email
- Slack/Teams
- documents
- structured systems (Jira/Linear)

The ontology would expand to include:

- projects
- tickets
- components
- customers
- teams

Extraction would link conversational claims to structured artifacts using:

- ticket references (IDs, URLs)
- entity linking to project metadata
- temporal co-occurrence signals

This would allow the system to connect informal discussions directly to operational workflows.

Long-Term Memory Behavior

In production, only stable and repeatable knowledge should become durable memory.
Durable memory would include:

- decisions
- recurring workflows
- ownership assignments
- persistent operational facts

Ephemeral context would include:

- brainstorming
- transient coordination
- incomplete planning discussions

To prevent drift, durable memory would require:

- multi-evidence support
- confidence thresholds
- temporal validation
- periodic reevaluation against new evidence

Grounding & Safety

Production deployment would require stricter provenance enforcement:

- every claim must reference exact source spans
- citations must remain resolvable over time
- evidence deletion must invalidate dependent claims

The system would track provenance chains so that redactions or deleted artifacts automatically update memory state.

Permissions

Layer10's environment requires strict access control.
Memory retrieval would be constrained by:

- user-level permissions
- artifact-level access rules
- inherited access from underlying systems

Because claims remain grounded to source artifacts, filtering can occur dynamically at query time.

Operational Reality

Scaling to production would require:

Incremental Updates

Streaming ingestion pipelines would replace batch processing, enabling:

- real-time updates
- idempotent reprocessing
- partial graph refresh

Cost Control

To manage compute costs:

- lightweight filtering would gate LLM usage
- embeddings could replace repeated LLM ranking
- caching would reduce recomputation

Evaluation & Monitoring

Production deployment would require:

- regression test suites
- retrieval accuracy tracking
- hallucination monitoring
- user feedback loops

These mechanisms ensure long-term reliability.

This system demonstrates a complete end-to-end memory pipeline for organizational knowledge: from structured extraction and grounded representation to retrieval, visualization, and practical deployment considerations. The design emphasizes traceability, correctness, and usability over scale, aligning closely with real-world workflows in enterprise environments. While lightweight, the architecture provides a strong foundation for extending Layer10's production setting across email, chat, documents, and structured operational systems.